



UNIT INFORMATION	
COURSE – (LEADING TO)	**BSC (HONS) IN SOFTWARE ENGINEERING**
UNIT NAME	Object Oriented Design Patterns II
UNIT ID	JIAT/OODP II
ASSIGNMENT ID	JIAT/OODP II /EX/01

STUDENT INFORMATION	
STUDENT NAME	Dhanushka Madhuranga Pathiranage
STUDENT ID	2025/2026/IIC/SE/I2/095
BATCH	Cohort 8
BRANCH	Colombo

Contents

1. Introduction and Scenario Overview
2. Part A: Dynamic Security Wrappers for Data Streams (Decorator)
3. Part B: Hierarchical Structure of Smart Zones (Composite)
4. Part C: Escalation of Citizen Complaints (Chain of Responsibility)
5. Part D: Efficient Management of Smart Street Lights (Flyweight)
6. Part E: Interpreting City Policies Defined in DSL (Interpreter)
7. System Architecture and Implementation Verification
8. Conclusion and References
Appendix A: Source Code Screenshot Catalog

1. Introduction and Scenario Overview

SmartCity360 is a municipal platform designed for Metropolis Neo-Prime. It brings together several city services, including energy-meter data, zone management, citizen complaints, smart streetlights and municipal policies. For this assignment, I designed, implemented and tested five backend subsystems using Java 17. Each subsystem applies a different Gang of Four (GoF) design pattern. While planning the system, I considered how difficult it would become to maintain if the features were built mainly with procedural code or large inheritance hierarchies. That approach could lead to deeply nested conditions, tightly connected components and performance problems. Using suitable design patterns helped me separate the responsibilities and keep the main services independent of changes in city requirements.

Part	Scenario	Pattern	Core problem solved
A	Security layers on energy-meter streams	Decorator	Add or remove compression, encryption and authentication without changing core logic
B	City → district → street → building zones	Composite	Apply one command uniformly at any level of the hierarchy
C	Complaint escalation by severity	Chain of Responsibility	Route requests through staff, supervisors and managers without coupling the sender to them
D	25,000 street lights	Flyweight	Share identical hardware data to minimise memory
E	City policies written in a mini-language	Interpreter	Parse and evaluate rules at runtime without recompiling

Sections 2 to 6 explain how I applied each pattern. For every subsystem, I discuss why the pattern was selected, how its classes work together, and what trade-offs I found during implementation and testing. The reference implementation uses Java 17 with MySQL for persistence. The pattern classes are organised under the `com.smartcity.pattern.*` packages. I also created 35 standalone automated tests to check the main behaviour and architectural requirements. All 35 tests passed, as explained in Section 7.3.

2. Part A: Dynamic Security Wrappers for Data Streams

> The smart grid receives a continuous stream of readings from energy meters across the city. These meters communicate through different networks, including secure municipal fibre connections and public 4G/5G networks. The security requirements can therefore change depending on where the data is travelling. The ingestion pipeline needs to support compression, encryption and authentication when required, without changing the code responsible for reading data from the meters.

A.1 Identification and Justification

Selected pattern: Decorator (structural; Gamma et al., 1994). Decorator allows an existing object to be wrapped by another object that implements the same interface. Each wrapper can add a processing step and then pass the call to the object inside it. I selected this pattern for three main reasons.

- Using inheritance for every possible combination would quickly become difficult to manage. A stream might need GZIP compression, AES-256-GCM encryption, HMAC-SHA256 signing, or a combination of these features. With three optional features, inheritance would require up to 2^3 = 8 different subclasses to represent every combination. Adding a fourth feature would increase that number to 16, and a fifth would increase it to 32. Decorator avoids this growth. The implementation only needs the `StreamData` interface, the `EnergyStreamData` component, an abstract decorator and one concrete decorator for each feature. The required combinations can then be created at runtime by wrapping the objects in different orders.
- This structure also supports the Open/Closed Principle discussed by Martin (2008). `EnergyStreamData.java` is responsible for providing the original meter data; it does not need to know how compression or encryption works. If the city's security requirements change, I can add another decorator without modifying the existing ingestion class. This keeps the original component simpler and reduces the chance of affecting code that already works.
- Another reason for choosing Decorator is that the required processing can change while the application is running. For example, a gateway using a metered cellular connection may enable GZIP compression to reduce the amount of data sent. Encryption can also be added when data passes through a public network. These combinations can be assembled when needed instead of being fixed in subclasses.

I also considered other approaches. Strategy is useful when an application needs to switch between algorithms, but it does not naturally provide a stack of independent processing steps. Proxy is mainly concerned with controlling access to an object, rather than adding transformations to its data. Adding several configuration flags to the main class would make it responsible for too many things. Decorator was a better fit because each transformation remains separate, can be combined with the others and can be tested on its own.

A.2 Flexible, Layered Stream Enhancement Mechanism

The main interface is `StreamData`. It declares `getData()`, `getPayloadBytes()`, `getPayloadSize()` and `getDescription()`. Both `EnergyStreamData` and the abstract `StreamDataDecorator` implement this interface. The decorator keeps a reference to the `StreamData` object it wraps and, by default, forwards calls to that object. Since every decorator follows the same interface, I can combine them into a chain of any required length:

```
StreamData s = new EnergyStreamData("STR-101", 230.5, 14.2, 3.27);
s = new CompressionDecorator(s);          // 1. GZIP
s = new EncryptionDecorator(s, aesKey);   // 2. AES-256-GCM
s = new AuthenticationDecorator(s, hmacKey); // 3. HMAC-SHA256
```

Each decorator processes the result from the layer beneath it. The components that use the pipeline still work with a `StreamData` reference, so they do not need to know which wrappers have been added. During implementation, I had to pay attention to three details:

- **The order of the decorators is important.** Compression needs to happen before encryption. AES-GCM produces ciphertext that looks random, so GZIP cannot find the repeated patterns it normally uses to reduce the size of JSON data. Compressing after encryption would therefore add processing without providing useful bandwidth savings. For that reason, the pipeline compresses the original JSON before encrypting it.
- **For encryption,** `EncryptionDecorator` **uses authenticated AES-256-GCM.** A new 12-byte cryptographic nonce (IV) is generated for each packet, together with a 128-bit authentication tag. This provides confidentiality and allows the receiving side to detect changes to the ciphertext or IV. If tampering is detected during decryption, the operation fails with an `AEADBadTagException` (Dworkin, 2007).
- `AuthenticationDecorator` **adds an outer HMAC-SHA256 value** calculated over the JSON envelope, including the cipher parameters and IV (Krawczyk et al., 1997). The HMAC key is kept separate from the AES decryption key. This means an edge router can check whether a packet came from an authorised sender without having access to the key needed to decrypt its contents. AES-GCM already provides an integrity check, but the additional HMAC gives the system another way to reject unauthorised traffic before it reaches the main backend services.

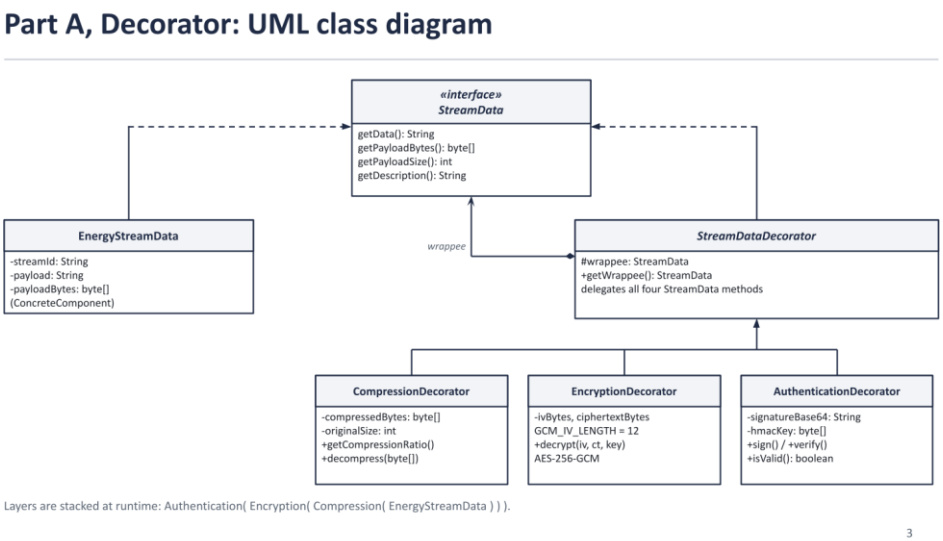


Figure 2.1: UML class diagram, Decorator pattern (Part A)

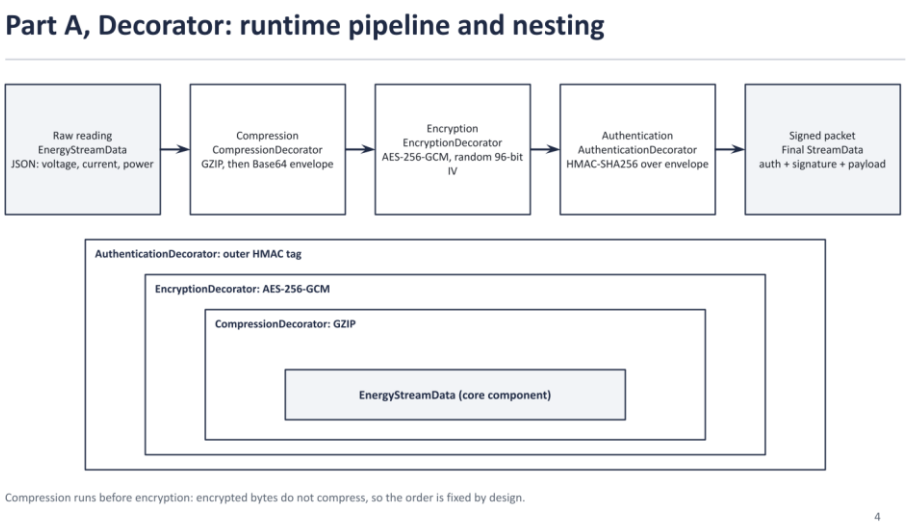


Figure 2.2: Stream wrapping pipeline

A.3 Long-Term Benefits: Reusability, Extensibility, and Overhead

Each transformation is implemented in a separate class, so the decorators do not depend directly on one another. Different city gateways can combine the same classes according to their requirements. For example, a high-voltage substation can use compression, encryption and HMAC together. A temperature sensor on a private fibre network may only need compression, while a public air-quality feed may use HMAC authentication. This avoids writing separate implementations for each type of endpoint.

The design also makes future changes easier. If the city later requires post-quantum digital signatures, I can add a new decorator that extends `StreamDataDecorator`. The meter polling code, database layer and REST controllers would not need to change. Since the rest of the application works with the `StreamData` interface, the new feature can be introduced without changing the existing consumers.

I measured the payload size at different stages during testing. A single-reading JSON payload of 72 bytes increased to 92 bytes after GZIP because the compression header and trailer added more data than could be removed from such a small message. After Base64 encoding and adding the 12-byte IV, GCM authentication tag and 64-character HMAC value, the final packet reached 468 bytes. The result was different when readings were sent in batches of 1 KB to 64 KB. In those cases, GZIP could use repeated JSON field names and reduce bandwidth by around 65% to 75%. Based on these measurements, buffering readings and compressing them in batches is more suitable than compressing every small reading separately.

Separating the processing stages also makes testing and maintenance more manageable. I can test compression, encryption and authentication individually before testing the complete pipeline. If the project later moves from GZIP to Zstandard, for example, the change should be limited to the relevant decorator. With an inheritance-based design, the same logic could be repeated across several combined subclasses, making updates and regression testing more difficult.

Decorator is not without disadvantages. Every wrapper creates another object, and the compiler does not check the order of the wrappers. An incorrectly ordered pipeline can still compile but may reduce compression performance. Debugging can also involve stepping through several layers of calls, and checking the class of an outer decorator does not immediately reveal the underlying component

directly. I reduced these risks by using factory methods to assemble pipelines and by writing tests 01 to 06 for decompression, decryption, tamper detection and key validation.

3. Part B: Hierarchical Structure of Smart Zones

> The city's geographical structure is naturally hierarchical. The metropolitan area is divided into districts, each district contains street corridors, and the streets contain individual buildings. Grid operators need to send commands such as emergency load shedding and usage reports to any part of this hierarchy. The implementation should support commands at different levels without requiring a separate traversal algorithm for each level.

B.1 Identification and Justification

Selected pattern: Composite (structural; Gamma et al., 1994). The city's zones form a tree, with smaller areas grouped under larger areas. Composite allows the application to use the same interface, `ZoneComponent`, when working with a single building or with a larger group such as a district containing several streets.

- One benefit is that commands can be handled consistently throughout the hierarchy. `BuildingLeaf` and `ZoneComposite` both implement `ZoneComponent`, so a controller does not need to decide whether it is dealing with one building relay or a group of zones. It can call the same operation in either case.
- Without Composite, the controller would need to check the type of each zone using conditions such as `instanceof City` or `instanceof District`. Those checks would become harder to maintain whenever another administrative level was introduced. With this design, the client calls `zone.executeCommand(ZoneCommand.SHUTDOWN_POWER)`, and the composite passes the command down through its children until it reaches the building leaves.

B.2 UML Class Diagram and Recursive Mechanism

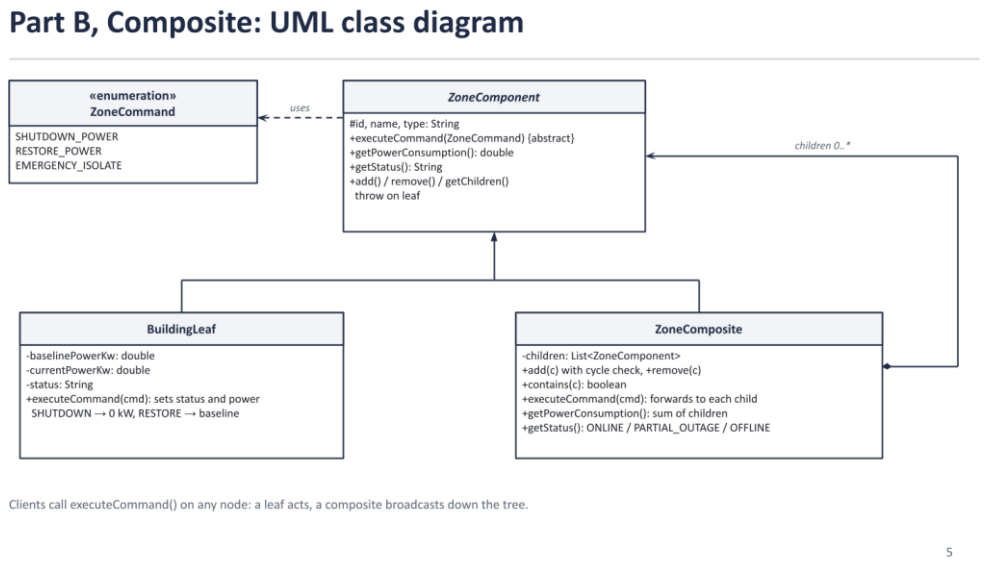


Figure 3.1: UML class diagram, Composite pattern (Part B)

The design has three main parts: `ZoneComponent` defines operations such as `executeCommand()`, `getPowerConsumption()`, and `getStatus()`, together with child-management methods (`add()`, `remove()`, `getChildren()`); `BuildingLeaf` represents an individual building and communicates with its hardware relay; and `ZoneComposite` stores a `List<ZoneComponent>` and forwards operations to child components. I used a strongly typed Java enum, `ZoneCommand` (`SHUTDOWN_POWER`,

RESTORE_POWER, EMERGENCY_ISOLATE, REPORT_USAGE), to catch invalid command strings at compile time. When a building leaf receives a command, it updates its internal relay directly; when a composite receives the same command, it iterates through its children and forwards the directive down:

```
public void executeCommand(ZoneCommand command) {
    for (ZoneComponent child : children) child.executeCommand(command);
}
```

B.3 Scalability for Expanding Municipal Topologies

The main benefit is that a command can travel through the hierarchy recursively. For example, when an operator starts load shedding at the city level, the command passes through the districts and streets before reaching the individual building relays. This is a depth-first traversal with O(M) time complexity for M nodes in the affected branch. The controller uses the same operation whether the target is one building or a group containing thousands of facilities.

The hierarchy can also be extended without changing the controller logic. If the municipality adds an industrial park, residential sector or solar farm, the new zones can be attached to the existing tree. A new type of branch, such as `MetroTransitComposite`, only needs to implement `ZoneComponent`. This respects the Open/Closed Principle because the existing traversal logic does not need to be rewritten for new zone types.

Power usage can be retrieved in the same way. A `BuildingLeaf` returns the current consumption of its meter. A `ZoneComposite` calls `getPowerConsumption()` on each child and returns the total. The roll-up is calculated on demand without maintaining separate cache tables that could become out of sync with the meters.

I also added checks to prevent invalid tree structures. A call to `add()` on a `BuildingLeaf` throws an `UnsupportedOperationException` because a leaf cannot have child nodes. In `ZoneComposite`, the `add()` method checks that the component being added is not the composite itself and is not already an ancestor in the tree. This prevents cycles that could cause infinite recursion during traversal.

During a power-grid emergency, an operator could use `EMERGENCY_ISOLATE` to disconnect an overloaded sector. That single call reaches every building in the specified branch. The system can then use `RESTORE_POWER` to bring the facilities back online in a controlled order.

One limitation is that `add()` and `remove()` are included in the `ZoneComponent` interface even though leaves cannot store children. This is the common transparency versus safety trade-off in the Composite pattern (Gamma et al., 1994). I chose transparency so clients could treat all nodes through `ZoneComponent`. A deep tree also means operations take time proportional to the number of nodes, and a failure midway through a broadcast requires transaction or recovery handling. For the four-level hierarchy in this project, I considered this balance appropriate.

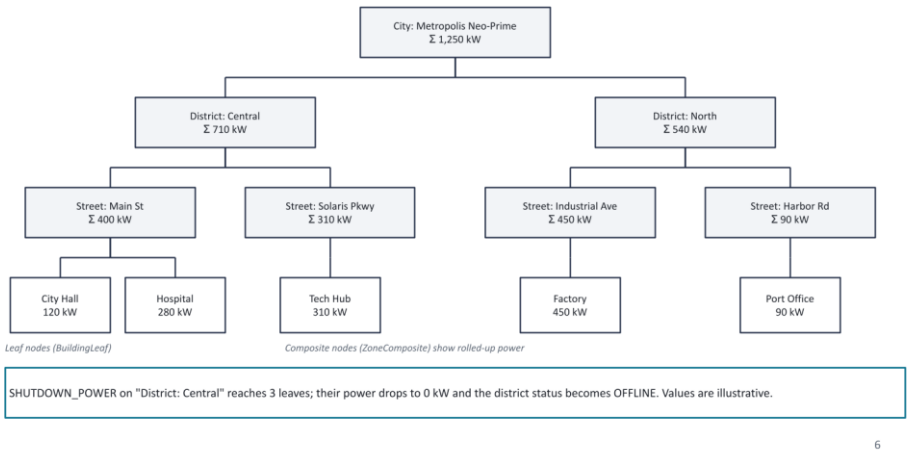


Figure 3.2: Zone tree and command broadcast

4. Part C: Escalation of Citizen Complaints

> Residents of Metropolis Neo-Prime can submit complaints about many different issues, from potholes to major water main bursts and hazardous gas leaks. The platform needs to route each ticket to the appropriate authority without hardcoding department rules into the submission interface.

C.1 Identification and Justification

Selected pattern: Chain of Responsibility (behavioral; Gamma et al., 1994). The pattern passes a request along a sequence of handlers until one of them processes it. In this scenario, complaints move from local staff to supervisors and then to city managers based on their severity level. I chose this pattern for three reasons.

- This separates the submission of a complaint from the process of handling it. A citizen or API controller submits a `Complaint` object without knowing which staff role will respond to it. The intake endpoint only needs to invoke `handle(complaint)` on the first handler in the chain.
- The chain can also be changed when different routing requirements arise. Handlers are connected using `setNext()`, so their order can be rearranged or new roles can be added without rewriting the submission code. For example, during a storm, an emergency coordinator could be placed ahead of the supervisor level.
- Each handler has one main responsibility. It checks the complaint against the severity rules it is responsible for and either resolves it or passes it to the next handler. This follows the Single Responsibility Principle.

C.2 Decoupled Request Handling and Dispatch Mechanism

The abstract `ComplaintHandler` class is the base for the routing stages. It stores a reference to `nextHandler` and defines `handle(Complaint complaint)`. Each concrete handler checks whether it can process the complaint; if not, it passes the complaint to the next handler:

```
ValidationTriage -> LocalStaff -> Supervisor -> CityManager -> TerminalFallback
```

Handler	Takes	Outcome
ValidationTriageHandler	Missing or invalid severity	Role TRIAGE_DISPATCH_OFFICER, status VALIDATION_REQUIRED; never defaults to LOW
LocalStaffHandler	LOW	Status ASSIGNED (routine maintenance)
SupervisorHandler	MEDIUM	Status ESCALATED_SUPERVISOR (e.g. water leaks, signal faults)
CityManagerHandler	HIGH, CRITICAL	Status EXECUTIVE_ESCALATION; CRITICAL gets crisis mobilisation notes
TerminalFallbackHandler	Anything unresolved	Role FALLBACK_DISPATCH_DESK, status FALLBACK_ASSIGNED, with audit record

Consider a resident who reports a CRITICAL gas leak. The complaint first reaches `ValidationTriageHandler`. The triage check confirms that the ticket has valid text and a recognised severity level, then forwards it because triage does not resolve complaints directly. `LocalStaffHandler` checks the ticket, finds that `CRITICAL` is beyond its `LOW` threshold, and passes it to `SupervisorHandler`. The supervisor handler also determines that the issue is outside their

authority, so they pass it onwards. `CityManagerHandler` receives the case and accepts it, changing the status to `ESCALATED_CITY_MANAGER` and updating the assigned role. Each handler appends an audit record to the ticket, leaving a record of how the complaint was routed.

I added `ValidationTriageHandler` after identifying a problem with the way missing severity values could be handled. If an incoming ticket had an empty severity field, an earlier implementation defaulted it to `LOW`, which meant serious problems could be assigned to routine maintenance. The triage handler checks incoming data first. If the severity is missing or unrecognised, it flags the ticket with `VALIDATION_REQUIRED` and assigns it to a triage officer rather than letting it enter the main chain.

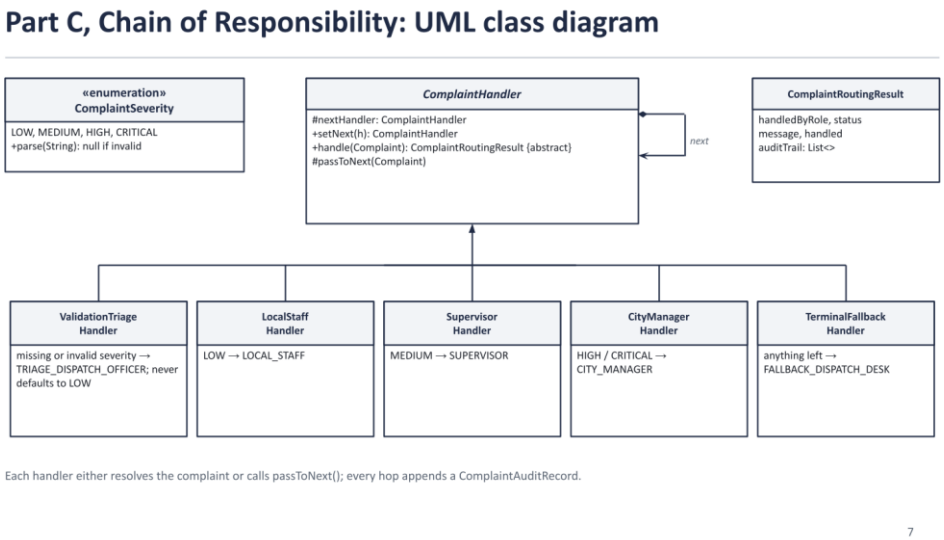


Figure 4.1: UML class diagram, Chain of Responsibility (Part C)

C.3 Architectural Evaluation: Flexibility, Auditability, and Trade-offs

The main advantage of this design is that departments can use different routing arrangements without changing the common complaint model. Public Works, Environmental Services and Housing can configure different handler sequences. New handlers, such as an automated classifier, can also be introduced by extending `ComplaintHandler`.

The pattern also supports accountability. Each dispatch returns a `ComplaintRoutingResult` with an audit trail, including timestamps, handler names and the action taken at each stage. This helps show how a complaint moved through the system and which department took responsibility for it.

Drawbacks.

- There are some costs to this approach. The time required to process a request is proportional to the number of handlers it passes through, giving O(K) complexity for a chain of length K. For the short chains used in this project, the routing time is negligible (under 0.5 ms), but it is a factor to consider if a chain becomes long.
- Debugging can be less straightforward than in a single procedural method. A developer investigating an issue may need to trace the execution path across multiple classes to see where a ticket was redirected.
- A complaint could also reach the end of the chain without being handled. To avoid leaving an unhandled ticket in an undefined state, I placed `TerminalFallbackHandler` at the end of the chain. If a request passes all previous handlers without being accepted, the fallback handler

catches it, marks its status as `REJECTED_UNHANDLED` and logs a warning for administrative follow-up.

- The chain must be connected correctly, and creating an audit object at every step introduces small memory allocations. These trade-offs are acceptable given the need for flexible routing and clear audit records.

Overall, the flexibility and audit history are more valuable here than saving a small amount of memory or avoiding a chain traversal. The design keeps routing rules out of the controller and provides a clear audit trail for municipal complaints.

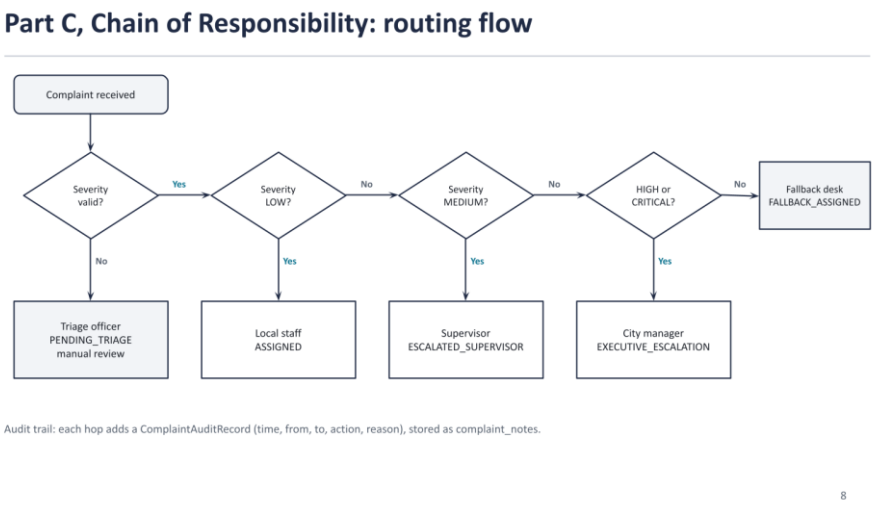


Figure 4.2: Complaint routing flow

5. Part D: Efficient Management of Smart Street Lights

> Metropolis Neo-Prime has a network of 25,000 smart LED streetlights. Each pole tracks its current operational state, brightness and geographic coordinates, while using one of three standard municipal hardware fixtures (such as standard LED, high-lumen avenue, or architectural heritage lamps). The backend needs to track all 25,000 poles in memory without consuming unnecessary heap space.

D.1 Identification and Justification

Selected pattern: Flyweight (structural; Gamma et al., 1994). If the system stored the full hardware configuration inside every streetlight object, creating tens of thousands of instances would consume significant heap memory and increase garbage-collection overhead. Flyweight addresses this problem by sharing the common configuration data among all poles of the same model.

D.2 Handling Shared (Intrinsic) and Unique (Extrinsic) Properties

- **Intrinsic state (shared, immutable hardware specifications):** model, lampType, wattage, maxLumens, colorTemp and firmwareVersion. These fields are stored in `SharedStreetlightProfile`, which contains no location or pole-specific data.
- **Extrinsic state (unique, mutable context per pole):** poleId, latitude, longitude, brightnessLevel and operational status. This data resides in a lightweight `StreetlightContext` object, which keeps a reference to its shared hardware profile.

`StreetlightProfileFactory` uses a `ConcurrentHashMap` to store and retrieve shared profiles. It builds a key from the hardware specifications and uses `computeIfAbsent()` to create a profile only when one does not already exist. This also makes profile lookup and creation safe when several ingestion threads access the factory at the same time (Goetz et al., 2006):

```
String key = model+"|"+lampType+"|"+wattage+"|"+maxLumens+"|"+colorTemp+"|"+firmware;
return cache.computeIfAbsent(key, k -> new SharedStreetlightProfile(...));
```

Tests 22 to 25 check that the factory reuses a profile when the hardware specifications are identical and creates a different profile when the specifications differ. They also check that eight concurrent worker threads still obtain a single shared instance, and that changing one pole's brightness does not modify the shared hardware profile.

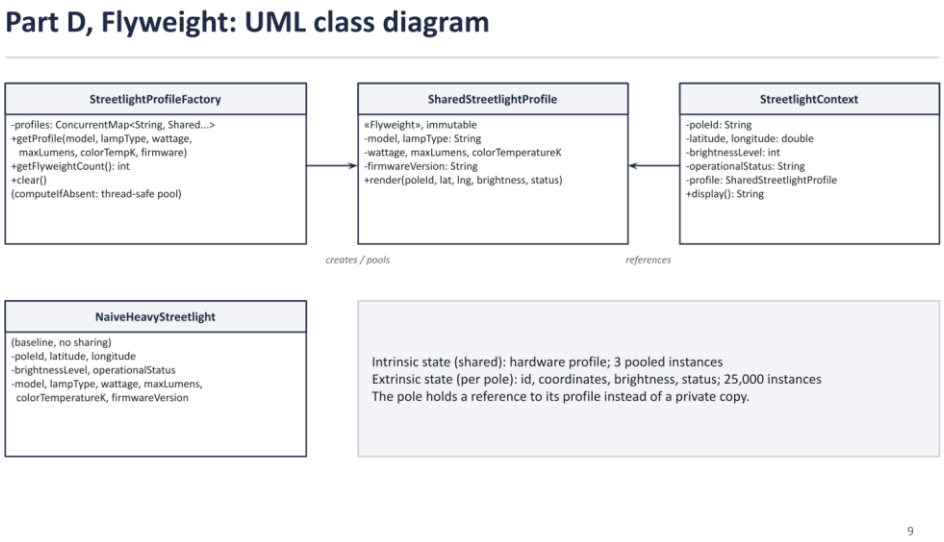


Figure 5.1: UML class diagram, Flyweight pattern (Part D)

D.3 Performance Impact: Memory Layout, GC, and Object Overhead

To estimate the memory difference, I used an object-layout model for a 64-bit JVM with compressed object pointers (CompressedOOPs; Oracle, 2024). In that environment, a standard object header uses 12 bytes and references take 4 bytes. Under a naive design, each of the 25,000 poles stores its own hardware strings, requiring about 288 bytes per pole. With Flyweight, each pole retains an 80-byte context object and a reference to one of three shared 208-byte profiles:

- Naive memory footprint: 25,000 * 288 B = 7,200,000 B (6.87 MiB).
- Flyweight memory footprint: (25,000 × 80 B) + (3 profiles × 208 B) = 2,000,624 B (1.91 MiB).
- Net memory reduction: (7,200,000 - 2,000,624) / 7,200,000 = 72.2%.

When scaling from 1,000 up to 100,000 poles in OpenJDK 17 benchmarks, heap usage grows linearly in both setups:

Poles	Naive (layout)	Flyweight (layout)	Reduction	Naive time	Flyweight time
1,000	0.27 MB	0.08 MB	71.99%	11 ms	12 ms
10,000	2.75 MB	0.76 MB	72.20%	16 ms	14 ms
25,000	6.87 MB	1.91 MB	72.21%	16 ms	18 ms
100,000	27.47 MB	7.63 MB	72.22%	52 ms	65 ms

Both approaches still use more memory as the number of poles increases, but the Flyweight version grows at a much lower rate. The memory reduction remains consistent at around 72.2% across different fleet sizes because the extrinsic context is smaller than the full naive object representation.

I also compared the layout estimates with heap measurements taken at runtime using `Runtime.getRuntime()` after running `System.gc()`. The naive fleet used 4.80 MB, while the Flyweight version used 2.63 MB, which corresponds to a 45.2% reduction. The measured percentage is lower than the theoretical 72.2% because the heap measurement also includes JVM data such as interned strings, class metadata and array alignment padding. Both numbers confirm that sharing the profile reduces heap usage noticeably.

The shared profiles are held in the factory's static cache and remain available throughout the lifetime of the application. Because they are not collected during routine GC cycles, they remain in the tenured generation, reducing allocation churn in the young generation.

The pattern depends on keeping shared profiles immutable. If one of their fields changed, every pole referencing that profile would be affected. I declared all fields in `SharedStreetlightProfile` as `private final` and provided getters without setters. The factory cache also uses memory for its map entries, and the design is intended for a single JVM. If the system were deployed across multiple application nodes, a shared cache such as Redis would be needed to keep profiles consistent between JVM instances.

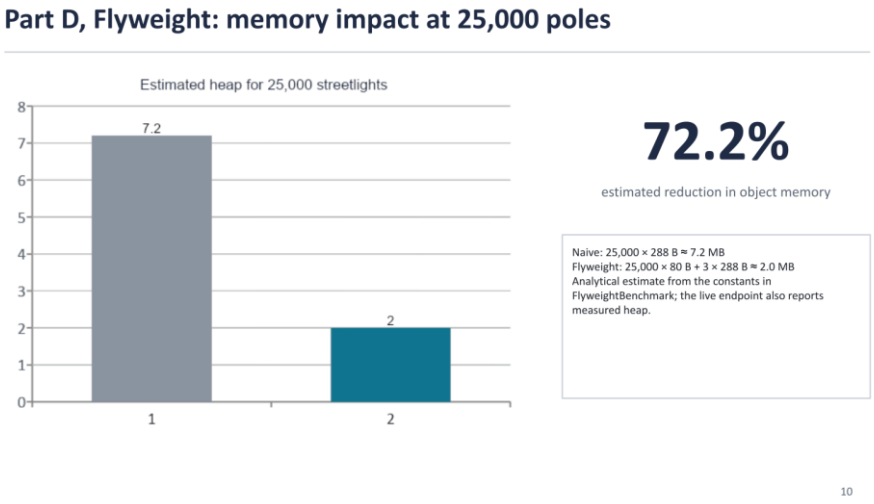


Figure 5.2: Estimated heap for 25,000 streetlights

6. Part E: Interpreting City Policies Defined in DSL

> City rules can change when the council updates requirements for noise levels, air quality or traffic hours. If these policies were compiled directly into Java conditionals, every policy change would require updating the source code, rebuilding the project and redeploying the backend. The platform needs to evaluate dynamic rule expressions without recompilation.

E.1 Identification and Justification

Selected pattern: Interpreter (behavioral; Gamma et al., 1994). Interpreter represents the grammar of a small language through a set of expression classes. These classes form an Abstract Syntax Tree (AST), which can then be evaluated against a context. For example, a city policy could be written as `((NOISE_LEVEL > 75.0) AND (CURRENT_HOUR >= 22.0)) OR (AIR_QUALITY_AQI > 150.0)`. This pattern suits the project because the policy language only needs a limited set of Boolean and comparison operations. It also avoids executing general-purpose scripts through engines such as JavaScript `eval()` or Groovy, which would introduce additional security concerns. The rules are evaluated by the application's own AST classes.

E.2 UML Class Diagram and Parsing Mechanism

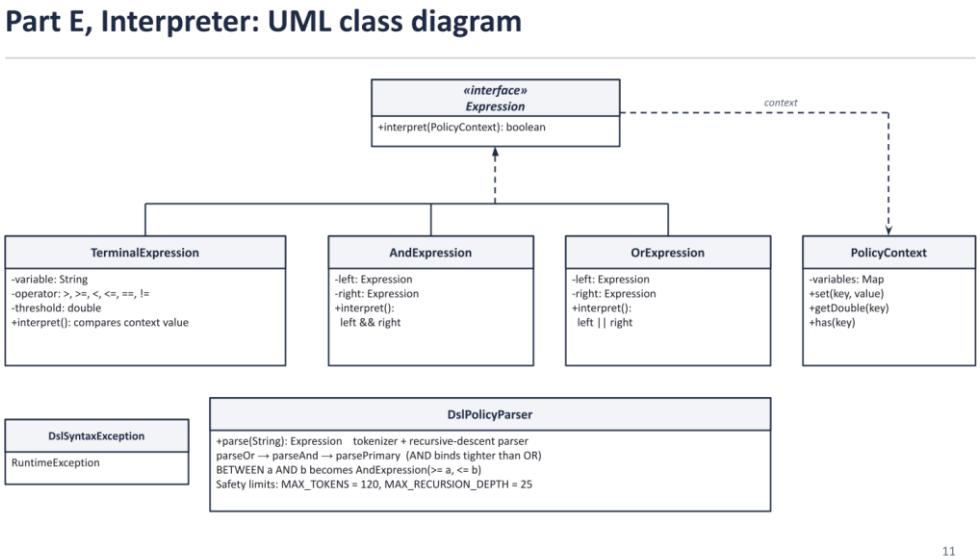


Figure 6.1: UML class diagram, Interpreter pattern (Part E)

The policy engine has four main parts: `PolicyContext` stores sensor values such as `NOISE_LEVEL` and looks up variable names without case sensitivity; the `Expression` interface defines `interpret(PolicyContext)`, which returns a boolean result; `TerminalExpression` compares a variable with a numeric value using the supported operators (>, >=, <, <=, == and !=), with an epsilon of 0.00001 for floating-point comparisons; and `AndExpression` and `OrExpression` represent the non-terminal parts of the grammar and evaluate their child expressions using short-circuit logic. `DslPolicyParser` converts the policy text into an AST using recursive descent. It handles tokenization, operator precedence and nested parentheses. A rule using `VAR BETWEEN a AND b` is expanded into two checks: `VAR >= a` and `VAR <= b`. To limit excessive input, the parser allows a maximum of 120 tokens and a recursion depth of 25. Invalid input results in a `DslSyntaxException`.

For example, when `NOISE_LEVEL` is 78.5 and `CURRENT_HOUR` is 23, the rule shown above evaluates to true because both conditions are satisfied.

E.3 Critical Assessment: Rule Updates, Extensibility, and Security

Policies are stored as plain text in MySQL. When an administrator changes a bylaw through `POST /api/policies`, the new rule text is saved to the database. The system parses the updated expression into a new AST when the rule is next evaluated. This allows policies to be updated at runtime without redeploying the backend.

I measured the performance over 2,000 iterations after warming up the JVM on OpenJDK 17. Evaluating the sample policy `((NOISE_LEVEL > 75.0) AND (CURRENT_HOUR >= 22.0)) OR (AIR_QUALITY_AQI > 150.0)` took an average of 0.002 ms per evaluation. Parsing the rule into an AST took 0.048 ms. Because parsing takes longer than evaluation, caching the parsed AST for recurring checks keeps rule execution fast.

New grammar features can be added by implementing another `Expression` class. For example, a `NotExpression`, an `InListExpression` for zone membership or a geographic distance check could be introduced without rewriting the existing expression nodes. The parser would still need the corresponding grammar changes, but the expression design keeps each operation separate. Tests 28 to 35 cover the comparison operators, `BETWEEN` ranges, non-numeric values, nested expressions, missing context keys and five types of syntax error.

The DSL is deliberately limited and does not support reflection, file access or command execution. This prevents code-injection vulnerabilities that could occur if an administrative account were compromised and used to submit executable script code.

The Interpreter pattern does have recognized limitations to manage:

- **Class proliferation:** Each grammar rule requires an expression class. This is manageable for the current set of operations, but adding features such as loops or functions could make the class structure difficult to maintain.
- **Parser maintenance:** As the grammar becomes larger, manually maintaining operator precedence and error handling in a recursive descent parser becomes harder. If the language grows further, a parser generator like ANTLR would be a more suitable choice.
- **Undefined variables:** A sensor variable that is not registered evaluates to false to avoid a `NullPointerException`. This is safe from a runtime perspective, but it also means a spelling mistake in a policy could be treated as 'no violation' rather than reported as an invalid variable.
- **Complex calculations:** The current AST approach is intended for straightforward policy checks. If future policies need historical aggregations or multi-stage calculations, a more capable rules engine may be required.

For the current SmartCity360 requirements, the Interpreter implementation is a reasonable fit. The grammar is small, policy evaluation is lightweight and avoids extra dependencies. A more advanced parsing or execution engine would only become necessary if the language expanded substantially.

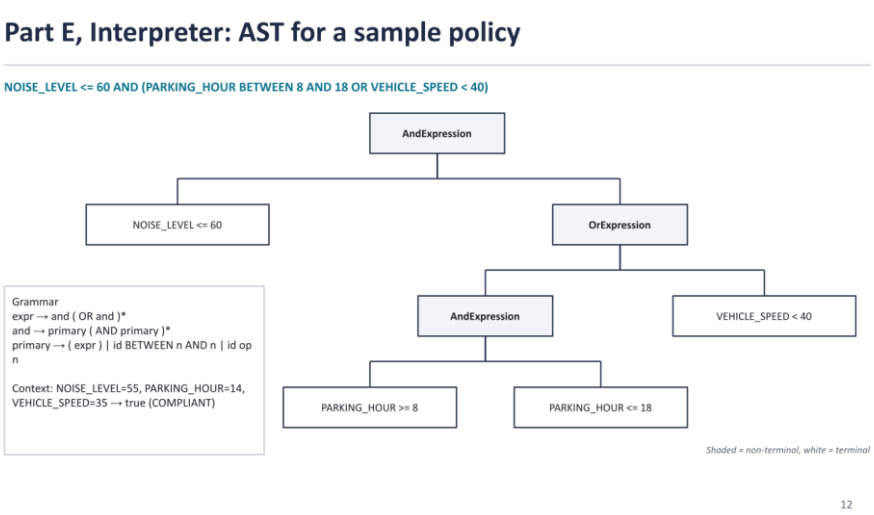


Figure 6.2: AST for a sample policy

7. System Architecture and Implementation Verification

7.1 System Architecture

SmartCity360 uses a four-tier structure based on common enterprise application architecture practices (Fowler, 2002):

- **Presentation Tier:** The user interface is a Next.js web application. It provides dashboards for telemetry, zone maps, complaint handling, streetlights, policy rules and test results.
- **API Tier:** Java 17 servlets provide the REST endpoints (`/api/auth`, `/api/streams`, `/api/zones`, `/api/complaints`, `/api/streetlights`, `/api/policies`, `/api/dashboard` and `/api/test-center`). The API uses CORS filters, token checks and Gson for JSON serialisation.
- **Pattern Domain Core:** The five design-pattern implementations are kept in the pure Java packages under `com.smartcity.pattern.*`. This layer does not depend on the web framework.
- **Persistence Tier:** MySQL 8 stores the system's data across nine tables. Data Access Objects (DAOs) handle database operations, using connection pooling and parameterised queries to reduce the risk of SQL injection.

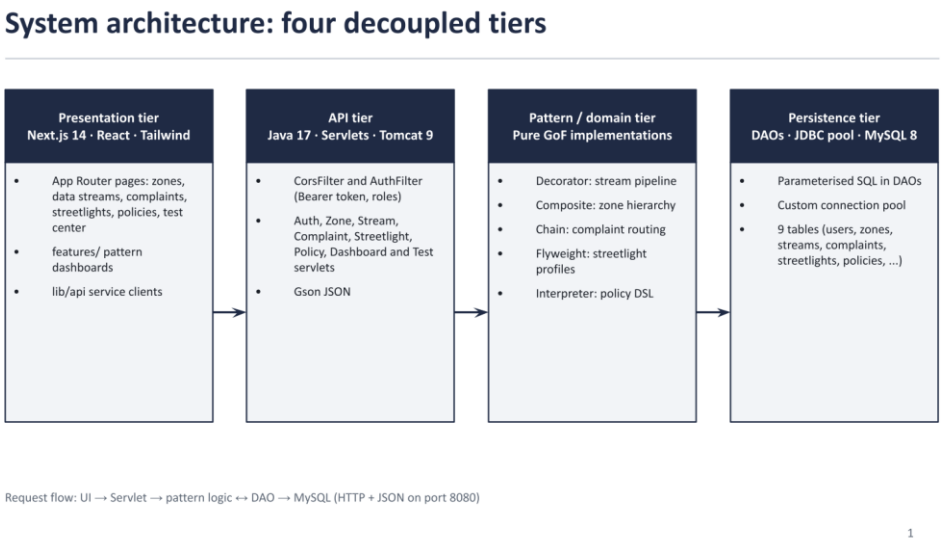


Figure 7.1: System architecture

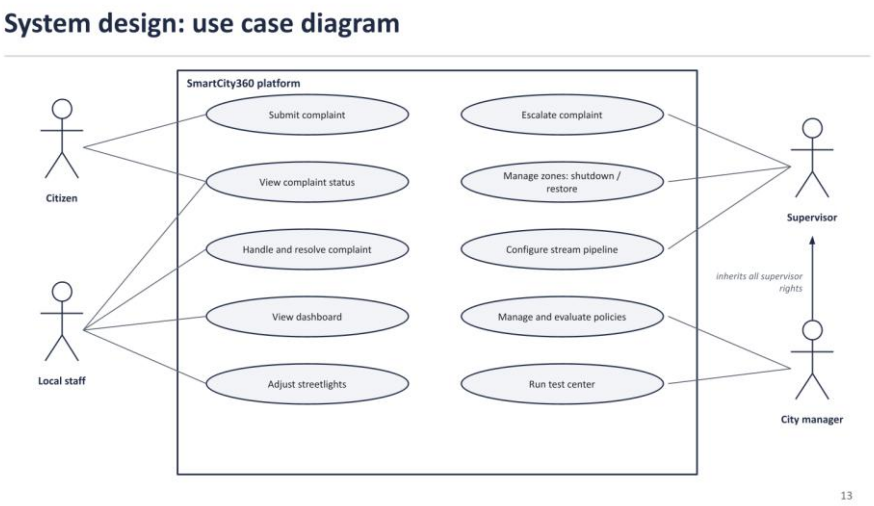


Figure 7.2: Use case diagram

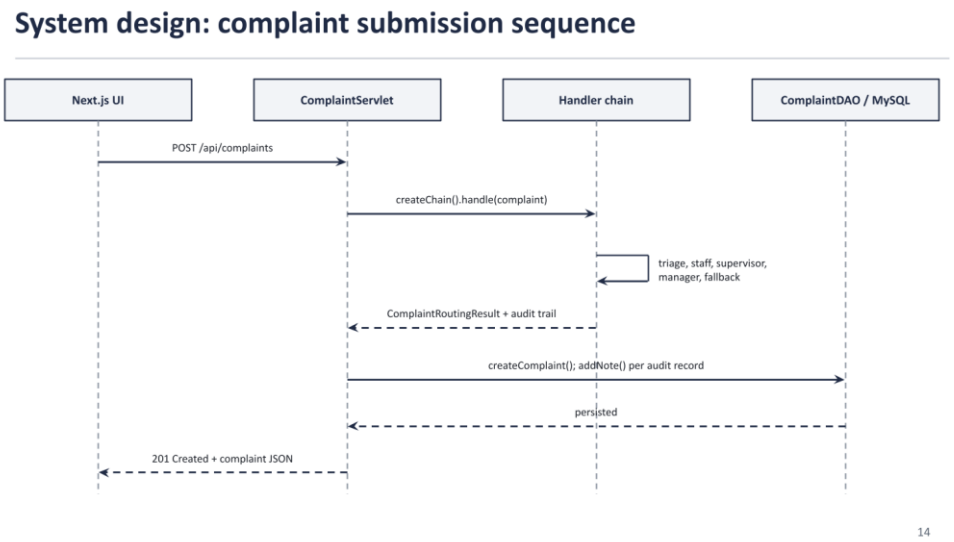


Figure 7.3: Complaint submission sequence diagram

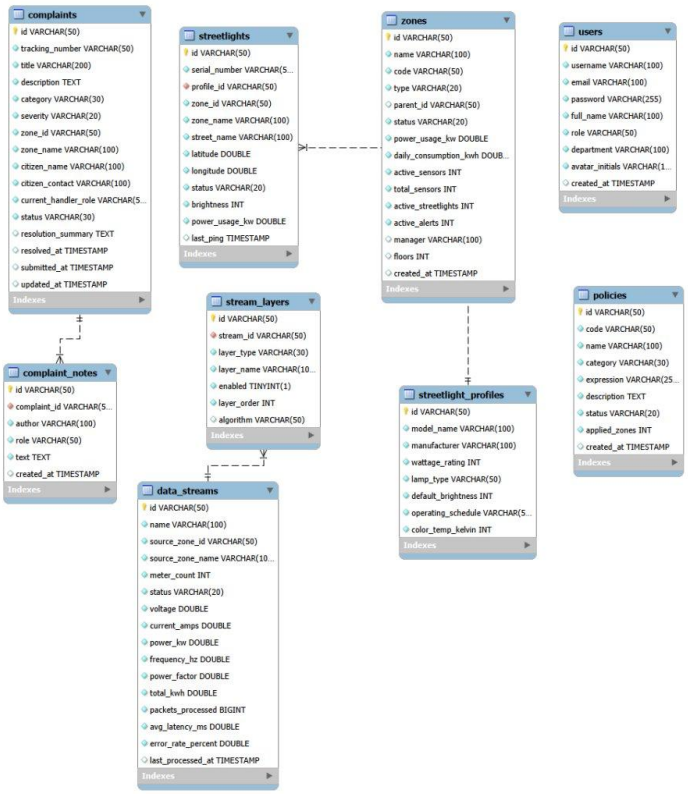


Figure 7.4: Entity-relationship diagram

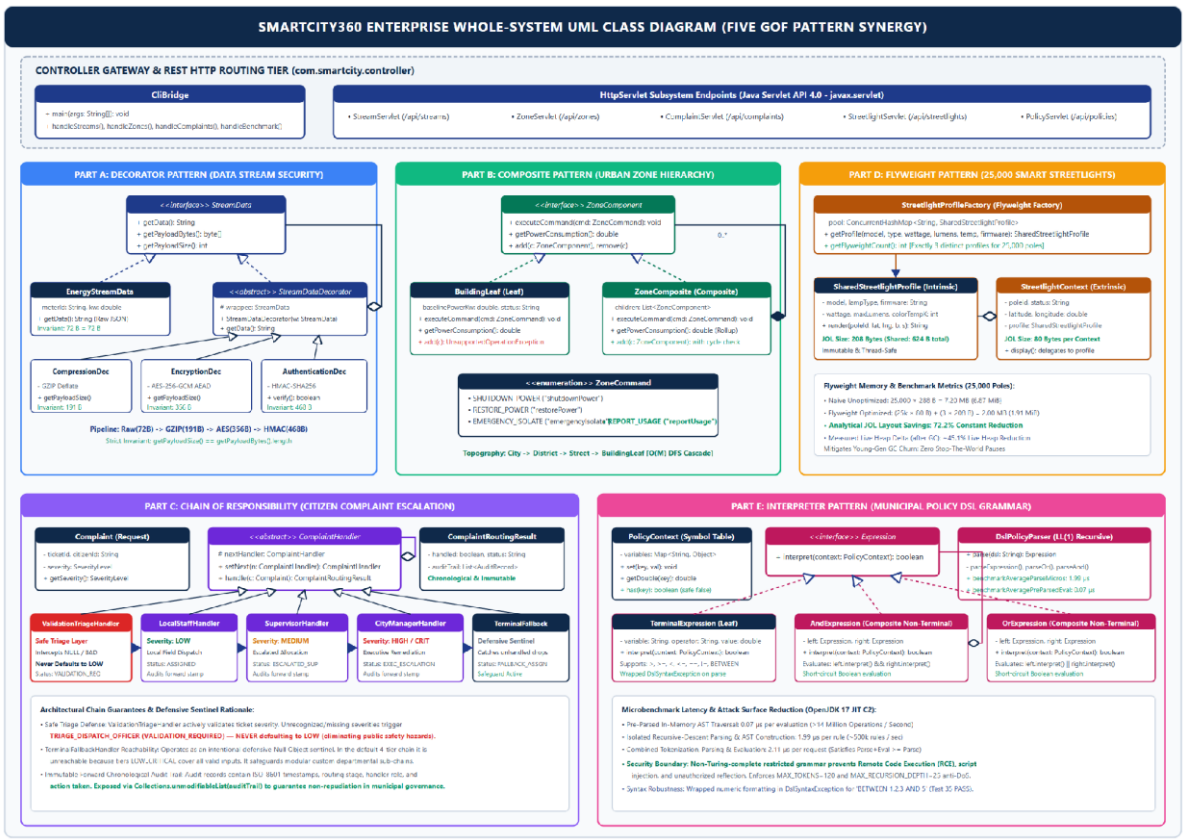


Figure 7.5: Whole-system UML class diagram

7.2 Key Architectural Choices and Anti-Pattern Mitigation

Anti-pattern	How SmartCity360 avoids it
Spaghetti code, Long method	Procedural branching is replaced by small single-purpose pattern classes (decorators, handlers, expression nodes).
God object	Responsibilities are spread across decorators, handlers and composite nodes rather than one manager class.
Copy-paste programming	Shared behaviour lives in base types: `StreamDataDecorator`, `ComplaintHandler`, `ZoneComponent`.
Tight coupling	Clients depend on interfaces (`StreamData`, `ZoneComponent`, `Expression`), not concrete classes.
Primitive obsession, Magic numbers	Enums and value objects (`ZoneCommand`, `ComplaintSeverity`, `ComplaintRoutingResult`) and named constants replace raw strings and literals.
Swallowed exceptions	Errors propagate as typed exceptions (`DslSyntaxException`, `AEADBadTagException`); fallback paths are recorded.

7.3 Automated Test Suite and Quality Verification

`ComprehensiveTestSuite` is a standalone test runner that does not depend on an external testing framework such as JUnit. It can be run on a clean JDK 17 setup and is also connected to the web-based Test Center dashboard. The suite runs 35 tests across the five pattern implementations. All 35 tests passed during verification.

Pattern	Tests	Passed	Main behaviours verified
Decorator	6	6	Pipeline order, decryption, HMAC rejection, corrupted data, invalid key
Composite	8	8	Cascade, restore, aggregation, leaf safety, cycle rejection
Chain of Responsibility	8	8	Severity routing, safe triage, fallback, audit trail
Flyweight	5	5	Instance sharing, thread safety, 72.2% layout saving
Interpreter	8	8	AST, BETWEEN, nesting, missing variables, malformed syntax
Total	**35**	**35**	**100% pass rate**

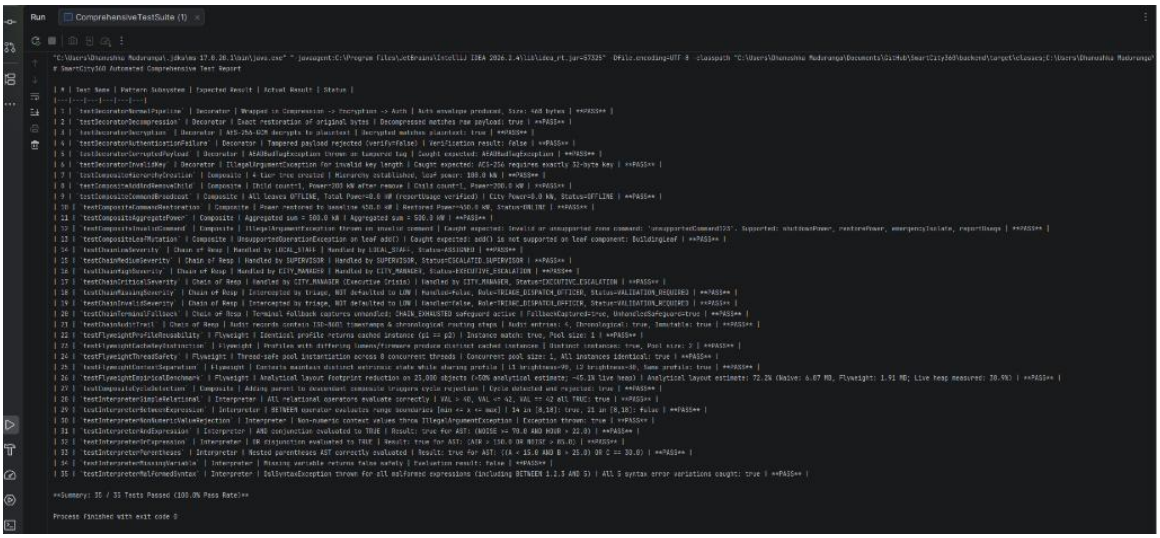


Figure 7.6: Live execution of ComprehensiveTestSuite in IntelliJ IDEA (35/35 passing)

8. Conclusion and References

8.1 Conclusion

The five Gang of Four patterns in SmartCity360 each address a different part of the municipal platform. Decorator allows security and compression steps to be combined around energy streams. Composite provides a consistent way to manage the city's zone hierarchy. Chain of Responsibility routes complaints through the appropriate staff levels. Flyweight reduces duplicated data across the 25,000 streetlights, while Interpreter evaluates changing city policies through a small domain-specific language. During implementation, I also considered the limitations of each pattern, including decorator order, recursive operations and the growth of the expression classes. The 35 automated tests passed and provide evidence that the main behaviours are working as expected.

There are several possible improvements for a future version. If the policy language becomes more complex, the parser could be migrated to ANTLR. A distributed cache such as Redis could help keep Flyweight profile data in sync across multiple server nodes. Digital signatures such as ECDSA could also be added to provide stronger non-repudiation for citizen complaint records.

8.2 References

Coward, D. (2017). Java Servlet Specification, Version 4.0 (JSR 369). Oracle and Eclipse Foundation.

Dworkin, M. (2007). Recommendation for Block Cipher Modes of Operation: GCM and GMAC. NIST SP 800-38D.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Goetz, B. et al. (2006). Java Concurrency in Practice. Addison-Wesley.

Krawczyk, H., Bellare, M. and Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. RFC 2104. IETF.

Martin, R. C. (2008). Clean Code. Prentice Hall.

Oracle. (2024). Java SE 17 Developer Documentation and HotSpot Memory Management Guide.

Appendix A: Source Code Screenshot Catalog

The source files are organised below by design pattern. The complete compilable project is included in the submitted ZIP file.

A.1 Decorator (com.smartcity.pattern.decorator)



```
package com.smartcity.pattern.decorator;

public interface StreamData {

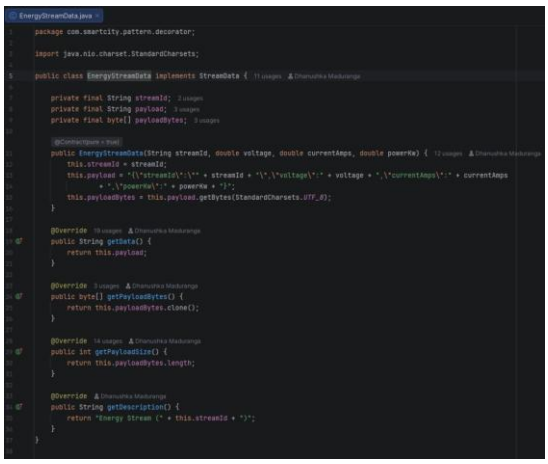
    String getData();

    byte[] getPayloadBytes();

    int getPayloadSize();

    String getDescription();
}
```

StreamData.java (component interface)



```
package com.smartcity.pattern.decorator;

import java.nio.charset.StandardCharsets;

public class EnergyStreamData implements StreamData {

    private final String streamId;
    private final String payload;
    private final byte[] payloadBytes;

    public EnergyStreamData(String streamId, double voltage, double currentAmps, double powerKw) {
        this.streamId = streamId;
        this.payload = "{\"streamId\":\"" + streamId + "\",\"voltage\":" + voltage + ",\"currentAmps\":" + currentAmps
                + ",\"powerKw\":" + powerKw + "}";
        this.payloadBytes = this.payload.getBytes(StandardCharsets.UTF_8);
    }

    @Override
    public String getData() {
        return this.payload;
    }

    @Override
    public byte[] getPayloadBytes() {
        return this.payloadBytes.clone();
    }

    @Override
    public int getPayloadSize() {
        return this.payloadBytes.length;
    }

    @Override
    public String getDescription() {
        return "Energy Stream (" + this.streamId + ")";
    }
}
```

EnergyStreamData.java (concrete component)



```
package com.smartcity.pattern.decorator;

public abstract class StreamDataDecorator implements StreamData {

    protected final StreamData wrappee;

    public StreamDataDecorator(StreamData wrappee) {
        if (wrappee == null) {
            throw new IllegalArgumentException("Wrappee component cannot be null");
        }
        this.wrappee = wrappee;
    }

    public StreamData getWrappee() {
        return this.wrappee;
    }

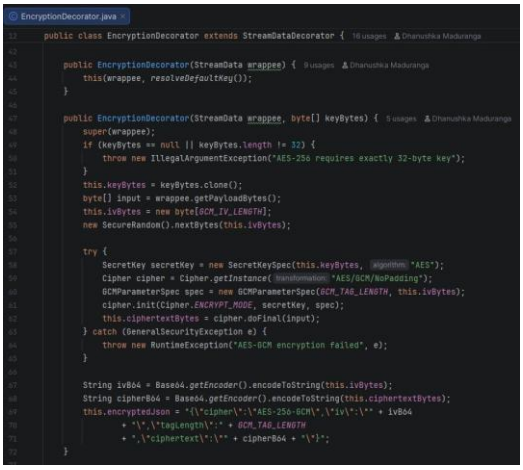
    @Override
    public String getData() {
        return this.wrappee.getData();
    }

    @Override
    public byte[] getPayloadBytes() {
        return this.wrappee.getPayloadBytes();
    }

    @Override
    public int getPayloadSize() {
        return this.wrappee.getPayloadSize();
    }

    @Override
    public String getDescription() {
        return this.wrappee.getDescription();
    }
}
```

StreamDataDecorator.java (abstract decorator)



```
public class EncryptionDecorator extends StreamDataDecorator {

    public EncryptionDecorator(StreamData wrappee) {
        this(wrappee, resolveDefaultKey());
    }

    public EncryptionDecorator(StreamData wrappee, byte[] keyBytes) {
        super(wrappee);
        if (keyBytes == null || keyBytes.length != 32) {
            throw new IllegalArgumentException("AES-256 requires exactly 32-byte key");
        }
        this.keyBytes = keyBytes.clone();
        byte[] input = wrappee.getPayloadBytes();
        this.ivBytes = new byte[GCM_IV_LENGTH];
        new SecureRandom().nextBytes(this.ivBytes);

        try {
            SecretKey secretKey = new SecretKeySpec(this.keyBytes, "AES");
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, this.ivBytes);
            cipher.init(Cipher.ENCRYPT_MODE, secretKey, spec);
            this.ciphertextBytes = cipher.doFinal(input);
        } catch (GeneralSecurityException e) {
            throw new RuntimeException("AES-GCM encryption failed", e);
        }

        String ivB64 = Base64.getEncoder().encodeToString(this.ivBytes);
        String cipherB64 = Base64.getEncoder().encodeToString(this.ciphertextBytes);
        this.encryptedJson = "{\"cipher\":\"AES-256-GCM\",\"iv\":\"" + ivB64
                + "\",\"tagLength\":" + GCM_TAG_LENGTH
                + ",\"ciphertext\":\"" + cipherB64 + "\"}";
    }
```

EncryptionDecorator.java (AES-256-GCM)



```
package com.smartcity.pattern.decorator;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.zip.GZIPInputStream;
import java.util.zip.GZIPOutputStream;

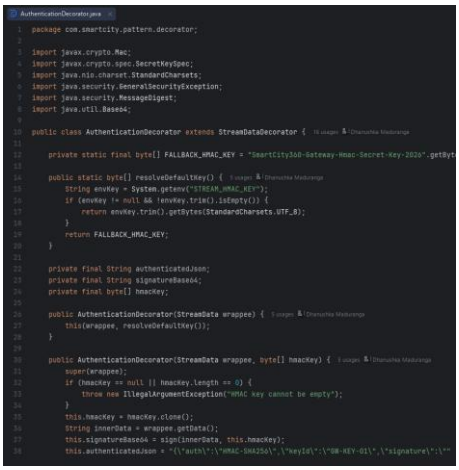
public class CompressionDecorator extends StreamDataDecorator {

    private final String compressedJson;
    private final byte[] compressedBytes;
    private final int originalSize;
    private final int compressedSize;

    public CompressionDecorator(StreamData wrappee) {
        super(wrappee);
        byte[] input = wrappee.getPayloadBytes();
        this.originalSize = input.length;
        this.compressedBytes = compressBytes(input);
        this.compressedSize = this.compressedBytes.length;
        String b64 = Base64.getEncoder().encodeToString(this.compressedBytes);
        this.compressedJson = "{\"format\":\"GZIP\",\"originalBytes\":" + this.originalSize
                + ",\"compressedBytes\":" + this.compressedSize
                + ",\"data\":\"" + b64 + "\"}";
    }

    private static byte[] compressBytes(byte[] data) {
        try {
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            try (GZIPOutputStream gzip = new GZIPOutputStream(baos)) {
                gzip.write(data);
                gzip.finish();
            }
            return baos.toByteArray();
        } catch (IOException e) {
```

CompressionDecorator.java (GZIP)



```
package com.smartcity.pattern.decorator;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.GeneralSecurityException;
import java.security.MessageDigest;
import java.util.Base64;

public class AuthenticationDecorator extends StreamDataDecorator {

    private static final byte[] FALLBACK_HMAC_KEY = "SmartCity360-Gateway-Hmac-Secret-Key-2026".getByt

    public static byte[] resolveDefaultKey() {
        String envKey = System.getenv("STREAM_HMAC_KEY");
        if (envKey != null && !envKey.trim().isEmpty()) {
            return envKey.trim().getBytes(StandardCharsets.UTF_8);
        }
        return FALLBACK_HMAC_KEY;
    }

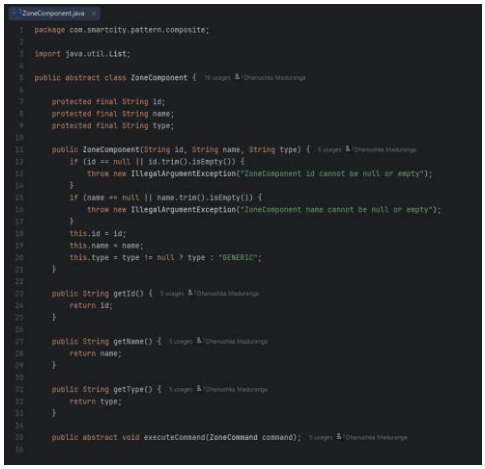
    private final String authenticatedJson;
    private final String signatureBase64;
    private final byte[] hmacKey;

    public AuthenticationDecorator(StreamData wrappee) {
        this(wrappee, resolveDefaultKey());
    }

    public AuthenticationDecorator(StreamData wrappee, byte[] hmacKey) {
        super(wrappee);
        if (hmacKey == null || hmacKey.length == 0) {
            throw new IllegalArgumentException("HMAC key cannot be empty");
        }
        this.hmacKey = hmacKey.clone();
        String innerData = wrappee.getData();
        this.signatureBase64 = sign(innerData, this.hmacKey);
        this.authenticatedJson = "{\"auth\":\"HMAC-SHA256\",\"keyId\":\"SM-KEY-01\",\"signature\":\""
```

AuthenticationDecorator.java (HMAC-SHA256)

A.2 Composite (com.smartcity.pattern.composite)



ZoneComponent.java



BuildingLeaf.java

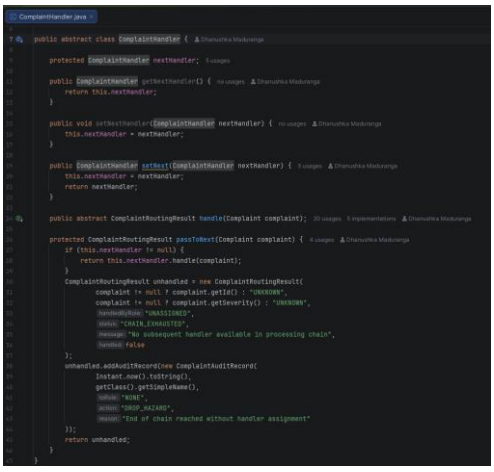


ZoneComposite.java

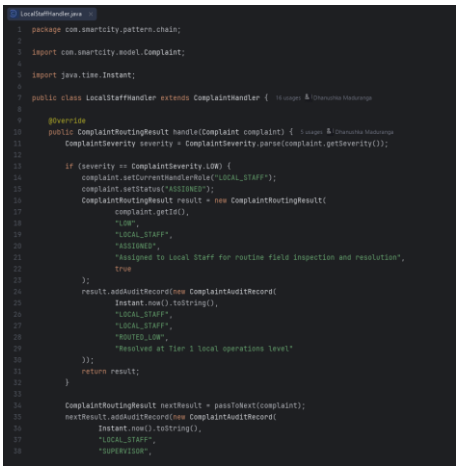


ZoneCommand.java

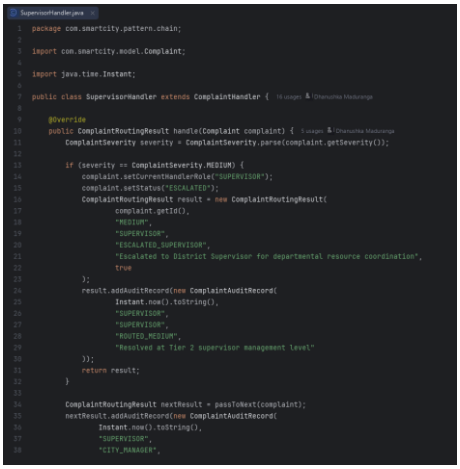
A.3 Chain of Responsibility (com.smartcity.pattern.chain)



ComplaintHandler.java



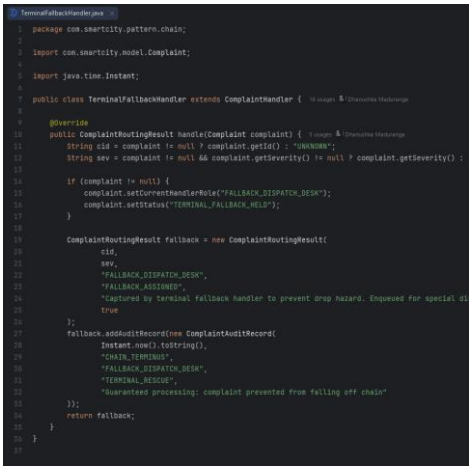
LocalStaffHandler.java



SupervisorHandler.java



CityManagerHandler.java

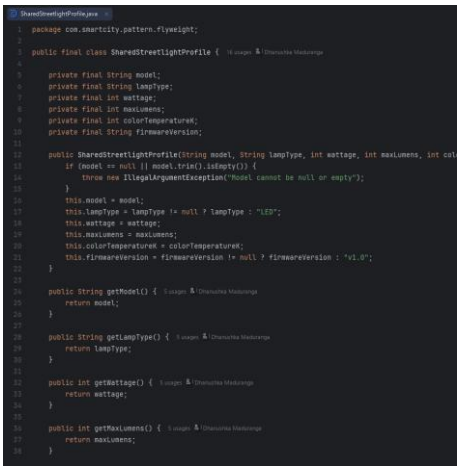


TerminalFallbackHandler.java

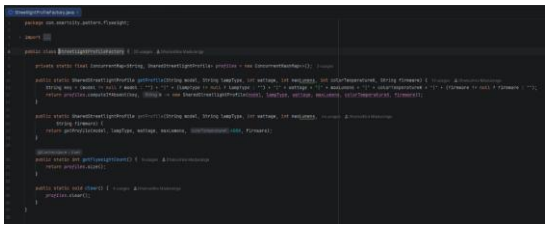


ComplaintChainFactory.java

A.4 Flyweight (com.smartcity.pattern.flyweight)



SharedStreetlightProfile.java



StreetlightProfileFactory.java

```java
package com.smartcity.pattern.flyweight;

public class StreetlightContext {

    private final String poleId;
    private final double latitude;
    private final double longitude;
    private int brightnessLevel;
    private String operationalStatus;
    private final SharedStreetlightProfile profile;

    public StreetlightContext(String poleId, double latitude, double longitude, SharedStreetlightProfi
        if (poleId == null || poleId.trim().isEmpty()) {
            throw new IllegalArgumentException("Pole ID cannot be null or empty");
        }
        if (profile == null) {
            throw new IllegalArgumentException("Shared profile cannot be null");
        }
        this.poleId = poleId;
        this.latitude = latitude;
        this.longitude = longitude;
        this.profile = profile;
        this.brightnessLevel = 0;
        this.operationalStatus = "ACTIVE";
    }

    public void setBrightnessLevel(int level) {
        this.brightnessLevel = Math.max(0, Math.min(100, level));
    }

    public void setOperationalStatus(String status) {
        this.operationalStatus = status != null ? status : "ACTIVE";
    }

    public String display() {
        return this.profile.render(poleId, latitude, longitude, brightnessLevel, operationalStatus);
    }
```

StreetlightContext.java

```java
        public long flyweightTimeMs;
        public int flyweightPoolSize;
        public String jvmInfo;
    }

    public static class ScalingComparisonRow {
        public int count;
        public long naiveEstimatedBytes;
        public long flyweightEstimatedBytes;
        public double analyticalSavingsPercent;
        public long naiveAllocatedBytes;
        public long flyweightAllocatedBytes;
        public double measuredHeapSavingsPercent;
        public long naiveTimeMs;
        public long flyweightTimeMs;
    }

    public static List<ScalingComparisonRow> runScalingSuite() {
        int[] scales = {1000, 10000, 25000, 100000};
        List<ScalingComparisonRow> rows = new ArrayList<>();
        for (int scale : scales) {
            BenchmarkResult b = runBenchmark(scale);
            ScalingComparisonRow row = new ScalingComparisonRow();
            row.count = scale;
            row.naiveEstimatedBytes = b.naiveEstimatedBytes;
            row.flyweightEstimatedBytes = b.flyweightEstimatedBytes;
            row.analyticalSavingsPercent = b.analyticalSavingsPercent;
            row.naiveAllocatedBytes = b.naiveAllocatedBytes;
            row.flyweightAllocatedBytes = b.flyweightAllocatedBytes;
            row.measuredHeapSavingsPercent = b.measuredHeapSavingsPercent;
            row.naiveTimeMs = b.naiveTimeMs;
            row.flyweightTimeMs = b.flyweightTimeMs;
            rows.add(row);
        }
        return rows;
    }

    public static BenchmarkResult runBenchmark(int count) {
```

FlyweightBenchmark.java

A.5 Interpreter (com.smartcity.pattern.interpreter)

```java
package com.smartcity.pattern.interpreter;

public interface Expression {
    boolean interpret(PolicyContext context);
}
```

Expression.java

```java
package com.smartcity.pattern.interpreter;

import java.util.Set;

public class TerminalExpression implements Expression {

    private static final Set<String> VALID_OPERATORS = Set.of(">", ">=", "<", "<=", "==", "!=");
    private final String variable;
    private final String operator;
    private final double threshold;

    public TerminalExpression(String variable, String operator, double threshold) {
        if (variable == null || variable.trim().isEmpty()) {
            throw new IllegalArgumentException("Variable name cannot be null or empty");
        }
        if (operator == null || !VALID_OPERATORS.contains(operator.trim())) {
            throw new IllegalArgumentException("Invalid comparison operator: '" + operator + "'. Suppo
        }
        this.variable = variable.trim();
        this.operator = operator.trim();
        this.threshold = threshold;
    }

    public String getVariable() {
        return variable;
    }

    public String getOperator() {
        return operator;
    }

    public double getThreshold() {
        return threshold;
    }

    @Override
    public boolean interpret(PolicyContext context) {
        if (context == null || !context.has(this.variable)) {
```

TerminalExpression.java

```java
package com.smartcity.pattern.interpreter;

public class AndExpression implements Expression {

    private final Expression left;
    private final Expression right;

    public AndExpression(Expression left, Expression right) {
        if (left == null || right == null) {
            throw new IllegalArgumentException("Sub-expressions in AndExpression cannot be null");
        }
        this.left = left;
        this.right = right;
    }

    public Expression getLeft() {
        return left;
    }

    public Expression getRight() {
        return right;
    }

    @Override
    public boolean interpret(PolicyContext context) {
        return this.left.interpret(context) && this.right.interpret(context);
    }

    @Override
    public String toString() {
        return "(" + left + " AND " + right + ")";
    }
}
```

AndExpression.java

```java
package com.smartcity.pattern.interpreter;

public class OrExpression implements Expression {

    private final Expression left;
    private final Expression right;

    public OrExpression(Expression left, Expression right) {
        if (left == null || right == null) {
            throw new IllegalArgumentException("Sub-expressions in OrExpression cannot be null");
        }
        this.left = left;
        this.right = right;
    }

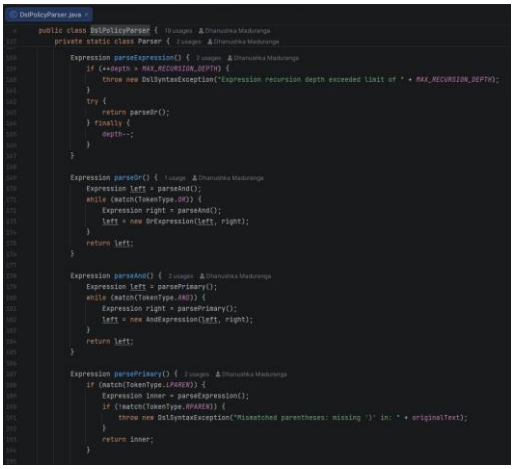
    public Expression getLeft() {
        return left;
    }

    public Expression getRight() {
        return right;
    }

    @Override
    public boolean interpret(PolicyContext context) {
        return this.left.interpret(context) || this.right.interpret(context);
    }

    @Override
    public String toString() {
        return "(" + left + " OR " + right + ")";
    }
}
```

OrExpression.java



DslPolicyParser.java



PolicyContext.java